

How I Built the DenseNet121 Model and Backend

Comparative Analysis of Deep Learning Models for Diabetic Retinopathy Severity Classification

DenseNet121

APTOS 2019

TensorFlow / Keras

Google Colab GPU

FastAPI

October 2026

In one sentence: I taught a computer to look at a photo of the back of the eye and say how serious the diabetic retinopathy is on a scale of 0 to 4, and then built a website where anyone can upload a photo and see the answer.

APTOS photos (Kaggle) → Google Drive → Split 70/15/15 → Crop + resize 224 → **DenseNet121** →
2-phase training (GPU) → Saved model → Test: 75.2% · κ 0.808 → FastAPI backend → **Website**

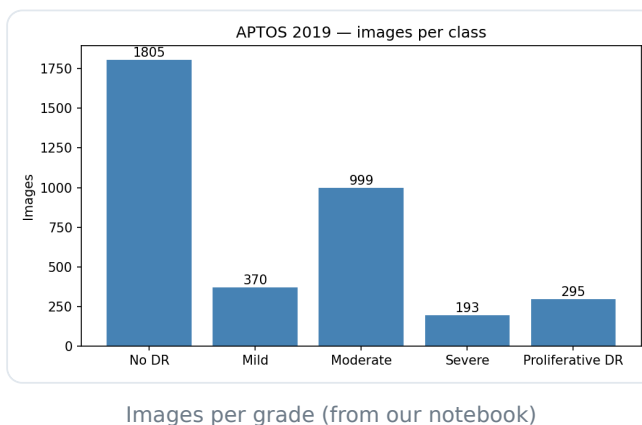
1 Getting the data

- I used the **APTOS 2019 Blindness Detection** dataset — a public Kaggle dataset of retinal photos from India.
- I created a Kaggle account, accepted the competition rules and generated an **API token** (a password for programs).
- I did not download it to my laptop. A **Google Colab notebook** downloaded it straight into **Google Drive** (8.6 GB).
- I kept only the **3,662 labelled photos**. Kaggle's test photos have no answers, so they cannot be used to measure accuracy.

Simple way to say it: Doctors had already graded 3,662 eye photos from 0 to 4. That is my "answer key" to learn from.

GRADE	MEANING	PHOTOS
0	No DR	1,805
1	Mild	370
2	Moderate	999
3	Severe	193
4	Proliferative DR	295

Key fact: the data is unbalanced — almost half the photos are healthy, only 193 are Severe.



2 Splitting the data

GROUP	PHOTOS	LIKE...
Train	2,564 (70%)	the textbook the model studies from
Validation	549 (15%)	practice tests during studying
Test	549 (15%)	the final exam — used only once, at the end

- **Stratified:** every group has the same mix of grades. **Fixed seed 42:** the split is identical every time.

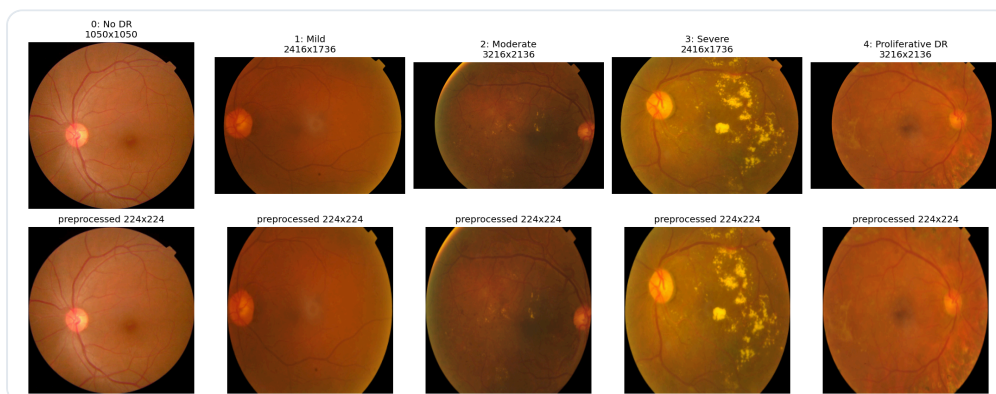
For teammates: ResNet and EfficientNet should use these **same split files** (Google Drive → [DR_project/splits/](#)), so all three models take the same exam.

3 Preparing the images (preprocessing)

The photos come from different cameras and sizes (1050 × 1050 up to 3216 × 2136 pixels). Before the model sees a photo:

1. **Colour fix** — OpenCV reads BGR, converted to normal RGB.
2. **Crop the black border** around the round eye — keep only the useful part.
3. **Resize to 224 × 224** — the size DenseNet was designed for.
4. **Normalise** pixel values the same way DenseNet was originally trained (ImageNet mean and std).

Simple way to say it: *Like making all exam papers the same size and format before marking them.*



One real image per grade — original (top) and after preprocessing (bottom)

- **Important:** the **same preprocessing code** is used for training and for the website — otherwise predictions would be wrong.
- **Training only — augmentation:** random flips, small rotations ($\pm 20^\circ$), zoom and brightness ($\pm 10\%$). More variety, so the model learns the disease instead of memorising photos.

4 The model — DenseNet121

- A neural network with **121 layers** that learns patterns: edges → shapes → signs of disease.
- Its special trick — **dense connections**: every layer receives the outputs of **all** earlier layers, not only the previous one.

Simple way to say it: In a normal network each student only reads the notes of the student before them. In DenseNet each student reads everyone's notes — so small details noticed early, like tiny red dots in the eye, are not lost.

- **Transfer learning:** DenseNet had already learned from 1.2 million everyday pictures (ImageNet). It already knows edges and shapes — I only taught it the eye-specific part.
- **What I added on top:** pooling → dropout (0.3) → a final layer with **5 outputs**, one probability per grade (they add up to 100%).
- Size: about **7 million parameters** — compact, which suits a small dataset.

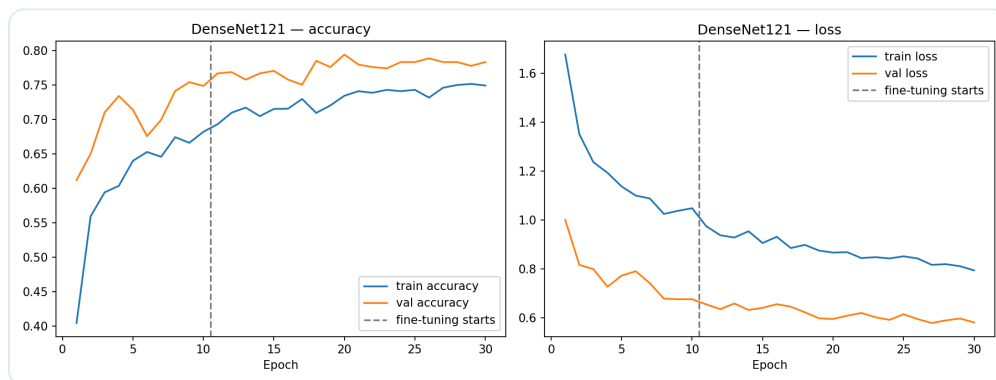
Photo 224×224×3 → DenseNet121 → 7×7×1024 features → pooling (1024) → dropout → 5 probabilities

5 Training (Google Colab T4 GPU, 17.7 minutes)

PHASE	WHAT LEARNS	LEARNING RATE	EPOCHS
1	Only my new final layer (DenseNet frozen)	0.001	10
2	Also DenseNet's last block, very gently	0.00001	20

Simple way to say it: First I trained only the new part so it would not spoil what DenseNet already knew. Then I let the last part of DenseNet adjust gently to eye photos.

- **Class weights:** mistakes on rare grades (like Severe) count more, so the model does not just answer "healthy" every time.
- The **best version** (lowest validation loss) was saved automatically as `densenet121_dr.keras` (47 MB).

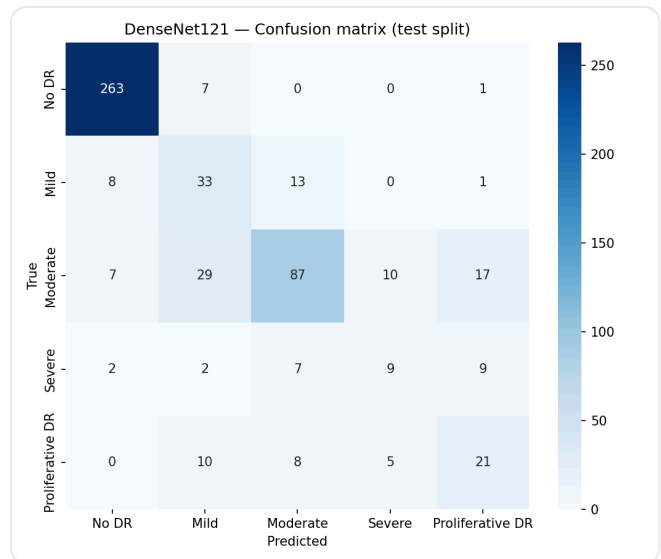


Accuracy and loss per epoch — dashed line: start of phase 2 (fine-tuning)

6 Results — the final exam (549 unseen photos)

METRIC	RESULT	MEANING
Accuracy	75.2%	about 3 out of 4 photos graded exactly right
Quadratic weighted kappa	0.808	official APTOS score; above 0.8 = strong — when wrong, usually off by just one grade
Macro F1	0.578	lower, because the rare grades are hard

- **Strongest:** healthy eyes — 97% found correctly.
- **Weakest:** Severe — 31% (few examples; looks like Moderate and Proliferative).



Confusion matrix — most mistakes are between neighbouring grades

Honest note: the model was still improving when the 20 fine-tuning epochs ended. More epochs could help, but **all three models must get the same number** to keep the comparison fair.

7 The backend (FastAPI)

The model on its own is just a file. The backend is a small server around it:

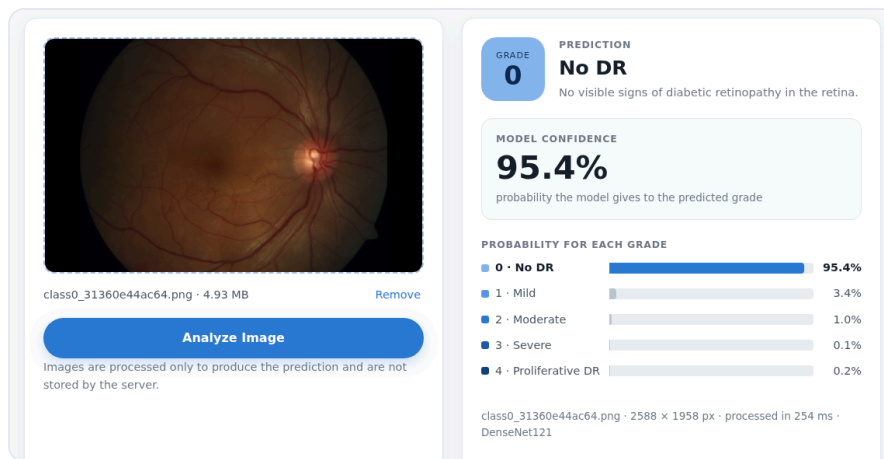
ADDRESS	WHAT IT DOES
GET /health	Is the server running? Is the model loaded?
GET /model-info	Details about the model, classes and preprocessing
POST /predict	Send a photo → get the grade, confidence and all 5 probabilities (JSON)

- Every upload is **checked first**: missing file, wrong type, corrupted image, too large → a clear error message instead of a crash.
- The model is loaded **once** at start-up; each prediction takes about **0.3 seconds** on a normal laptop CPU.

Simple way to say it: The backend is like a receptionist: it checks your file, passes it to the model, and gives you back the answer in a standard format.

8 The website

- Explains diabetic retinopathy, the five levels and how the model works.
- **Upload a photo → see the prediction** with a bar for every grade.
- Shows our **real results**; ResNet and EfficientNet are added when their results are ready.



The website's AI Analysis section with a real prediction

9 Testing

- **39 automated tests** — preprocessing, the model, the full training pipeline and every backend error case — all passing.
- A browser test confirmed that what the website shows is **exactly** what the model outputs.
- Predictions on my laptop are identical to the predictions made on Colab.

For a fair comparison, please keep the same

1. **Split files** — DR_project/splits/ (train / val / test CSVs)
2. **Image size and preprocessing** — crop the black border, resize to 224 × 224
3. **Training settings and metrics** — accuracy, quadratic weighted kappa, macro F1, confusion matrix

Full details: [docs/common_experiment_settings.md](#) · Code: github.com/ShaikNousheen1700/diabetic-retinopathy-member3

Research prototype for academic purposes — not a medical diagnosis.